UNIVERSIDAD SIGLO 21

Seminario Práctico de Informática

Módulo 4

Trabajo Práctico 4



Caso de estudio: ReperTrack - Sistema Integrado de Gestión de Repertorio y Práctica Musical

Estudiante: Juan Martín Canguillen
DNI: 44933871
Profesor Titular: Pablo Alejandro Virgolini
Carrera: Licenciatura en Informática
Año: 2026 - 2.° cuatrimestre
Fecha de entrega: 17 de octubre de 2026

Índice

Parte I. Trabajo Práctico 1 — Inicio del PUD

1. Introducción y justificación

2. Objetivos del proyecto y del sistema

3. Alcance y elicitación

4. Diagrama de dominio

5. Propuesta funcional y técnica

6. Requerimientos y casos de uso

Parte II. Trabajo Práctico 2 — Análisis y diseño

7. Análisis y secuencia de diseño

8. Diagrama de clases de diseño

9. Despliegue y aclaración del CU011

10. Plan de pruebas (CP01..CP18)

11. Base de datos, DER y scripts SQL

12. Prototipos de consola y comunicaciones

Parte III. Trabajo Práctico 3 — Prototipo Java

13. Los cuatro pilares de la POO

14. Excepciones y evidencia de ejecución

15. Menú, estructuras propias y persistencia JDBC

Parte IV. Trabajo Práctico 4 — Entrega integradora

16. Patrón de diseño MVC

17. Arreglos y ArrayList

18. Manejo de archivos de texto

19. Excepciones en la interacción con MySQL

20. Clases abstractas e interfaces

21. Lógica de negocio y reglas RN-01..RN-05

22. Capturas del programa y video

23. Correcciones aplicadas y cierre

24. Participación activa

Anexo A. Diagrama de clases completo

Anexo B. Capturas de ejecución

Anexo C. Consultas SQL

Anexo D. Guion del video

Referencias bibliográficas

Repositorio público del proyecto: https://github.com/duendecillo2/reper-track

Parte I. Trabajo Práctico 1 — Inicio del PUD

Esta primera parte retoma el documento original del TP1 con las correcciones de redacción y de modelo que quedaron pendientes. Se mantienen el problema, la justificación y el alcance; se reescriben los objetivos y se ajusta el diagrama de dominio al relevamiento de procesos.

1. Introducción y justificación

La práctica musical de nivel intermedio o avanzado exige organizar, en paralelo, un repertorio vivo, una biblioteca de ejercicios técnicos y metas de mediano y largo plazo. En la mayoría de los casos ese control se hace de forma artesanal: cuadernos, planillas dispersas o la memoria del intérprete. El problema no es nuevo —la literatura sobre práctica deliberada de Ericsson, Krampe y Tesch-Römer (1993) ya lo señalaba—, pero sigue sin una respuesta informática adecuada para el músico individual.

ReperTrack nace de esa experiencia personal: durante años llevé mi repertorio en cuadernos que se perdían y en planillas que abandonaba por cansancio. El sistema propone un repositorio único, un cronómetro integrado y métricas de progreso que permitan decidir qué practicar con evidencia y no con intuición. La justificación es, a la vez, técnica (Java + MySQL + JDBC) y personal (un músico que necesita la herramienta).

2. Objetivos del proyecto y del sistema

Objetivo general del proyecto. Desarrollar ReperTrack, una aplicación de consola en Java 11 con persistencia en MySQL 8 y arquitectura MVC, que permita a músicos intermedios y avanzados gestionar su repertorio y cronometrar sesiones de práctica vinculadas a instrumentos y elementos de estudio, generando estadísticas de progreso que respalden la planificación de metas.

Objetivos específicos.

- Implementar el ABM de Instrumentos y de Elementos de Estudio (Canciones y Ejercicios), con persistencia transaccional y validación mediante excepciones personalizadas.
- Desarrollar el registro cronometrado de sesiones, con iniciar/pausar/finalizar, asociado obligatoriamente a un Instrumento y a un Elemento.
- Implementar el módulo de estadísticas: acumulados por instrumento y por elemento, frecuencia semanal y promedio por sesión.
- Aplicar el patrón MVC separando menú de consola, controladores/servicios y modelo/DAO.
- Construir un esquema relacional en 3FN con claves foráneas, UNIQUE, CHECK e índices compuestos.
- Versionar el código y los scripts en un repositorio público de GitHub con README de instalación y ejecución.

Objetivo general del sistema. Registrar, almacenar y procesar los datos de la práctica musical —instrumentos, elementos y sesiones cronometradas— para producir información agregada de progreso que respalde la toma de decisiones sobre qué obras ensayar, cuánto tiempo dedicar a cada una y qué elementos retirar del repertorio activo.

3. Alcance y elicitación

El prototipo cubre el ABM de instrumentos y elementos, el registro cronometrado de sesiones, la consulta y eliminación de sesiones, las estadísticas de progreso, la exportación de reportes (PDF y TXT) y las alertas de mantenimiento. No contempla reproducción de audio, sincronización en la nube, multiusuario ni autenticación remota: el acceso físico al equipo es la frontera de seguridad.

La elicitación se apoyó en la experiencia directa del autor como músico y en la observación de las herramientas que usa a diario (cuaderno, Excel, notas del celular). El «cliente» de esta primera

versión es el propio músico, lo que elimina la curva de aprendizaje del dominio y permite validar cada módulo con la rutina real de práctica.

4. Diagrama de dominio

El diagrama se ajustó para mostrar atributos, multiplicidades y la asociación directa entre Músico y SesiónDePráctica, que en la versión original quedaba implícita. Músico posee instrumentos y registra sesiones; Instrumento agrupa elementos y se asocia a sesiones; ElementoDeEstudio se aborda en sesiones y se especializa en Canción y Ejercicio.

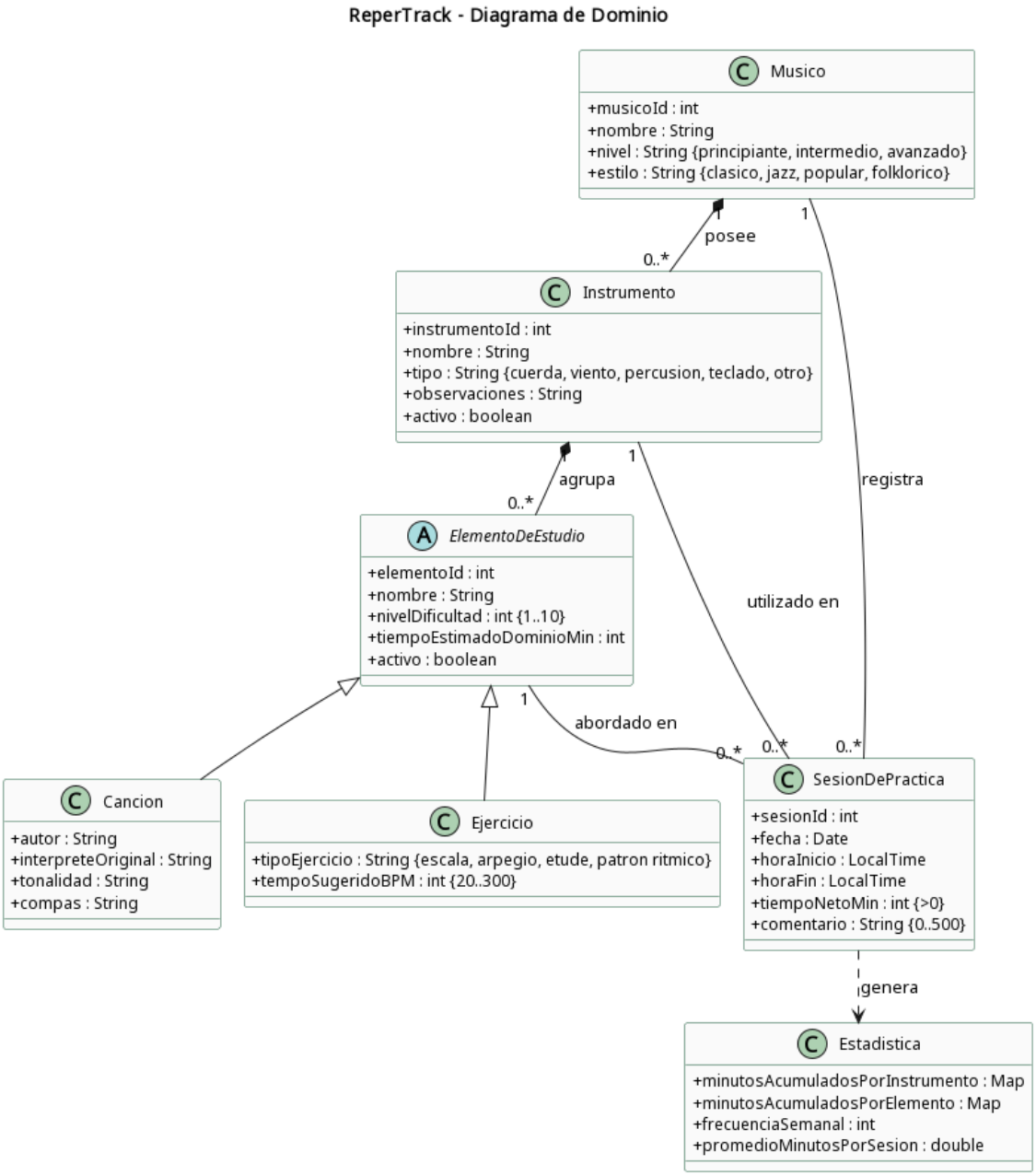


Figura 1. Diagrama de clases de dominio (corrección del TP1).

5. Propuesta funcional y técnica

Propuesta funcional. Cuatro módulos: gestión de instrumentos y elementos, registro transaccional de sesiones, estadísticas y reportes, y alertas de mantenimiento. Cada módulo se corresponde con un caso de uso y con un conjunto de reglas de negocio.

Propuesta técnica. Java 11 LTS, MySQL 8.0 (InnoDB, utf8mb4), JDBC oficial (mysql-connector-j), Maven como gestor de dependencias y una aplicación de consola pura (Scanner + System.out) como capa de presentación. No se usan frameworks de UI ni de persistencia: la consigna pide Java y JDBC, y la consola mantiene el prototipo liviano, repetible y fácil de probar.

6. Requerimientos y casos de uso

Los requerimientos funcionales se numeraron RFS01..RFS11 y los no funcionales RNF01..RNF08. Los primeros cubren ABM, cronómetro, asociación obligatoria, listado, eliminación, estadísticas, alertas y exportación; los segundos fijan Java 11, MySQL 8, MVC, GitHub, transacciones ACID, usabilidad, portabilidad y pruebas.

Los actores son el Músico (principal), el Sistema de Persistencia (secundario), el Reloj del Sistema y el Sistema de Archivos. El caso de uso crítico es CU006 (Registrar Sesión de Práctica), que incluye la validación de integridad referencial (CU010) y puede disparar el recálculo de alertas (CU012). CU011 (Exportar Reporte) extiende a CU009 (Consultar Estadísticas).

Requerimientos funcionales

ID	Descripción
RFS01	ABM de instrumentos (nombre, tipo, observaciones).
RFS02	ABM de canciones (autor, intérprete, tonalidad, compás, dificultad).
RFS03	ABM de ejercicios (tipo, BPM, dificultad).
RFS04	Cronómetro con iniciar, pausar y finalizar.
RFS05	Asociación obligatoria de la sesión a instrumento y elemento.
RFS06	Listado y filtrado de sesiones por instrumento, elemento y fecha.
RFS07	Eliminación de sesión con confirmación explícita.
RFS08	Tiempo acumulado por instrumento y por elemento.
RFS09	Frecuencia semanal y promedio por sesión.
RFS10	Alertas de mantenimiento por inactividad.
RFS11	Exportación del reporte a PDF y TXT.

Requerimientos no funcionales

ID	Categoría	Descripción
RNF01	Implementación	Java 11 o superior, aplicación de consola.
RNF02	Persistencia	MySQL 8 con JDBC oficial.
RNF03	Arquitectura	Patrón MVC con capas separadas.
RNF04	Calidad	Repositorio público en GitHub con README.
RNF05	Confiabilidad	Transacciones ACID con commit/rollback.
RNF06	Usabilidad	Mensajes claros y confirmación en eliminaciones.
RNF07	Portabilidad	Windows, Linux y macOS sin recompilar.
RNF08	Mantenibilidad	Pruebas automatizadas del dominio y los DAO.

Parte II. Trabajo Práctico 2 — Análisis y diseño

El TP2 se corrigió en los puntos que el docente señaló: la secuencia de diseño ahora usa exclusivamente las clases del diagrama de clases; ElementoDeEstudio guarda una asociación a Instrumento (no un id suelto); el despliegue tiene definiciones completas; el CU011 está especificado; la nomenclatura de pruebas quedó unificada; el DER muestra tipos SQL y las pruebas de integridad se ejecutan en MySQL.

7. Análisis y secuencia de diseño

El análisis se documentó con clases de interfaz, control y entidad. La secuencia de diseño del CU006 usa exactamente las clases que luego se programaron: MenuPrincipal, SesionController, SesionService, SesionDePractica, Instrumento, ElementoDeEstudio, SesionDAO, ConexionDB y MySQL. El flujo normal pide instrumento y elemento, valida su existencia, construye la sesión, la valida (RN-01) y la inserta dentro de una transacción (setAutoCommit(false), commit, rollback).

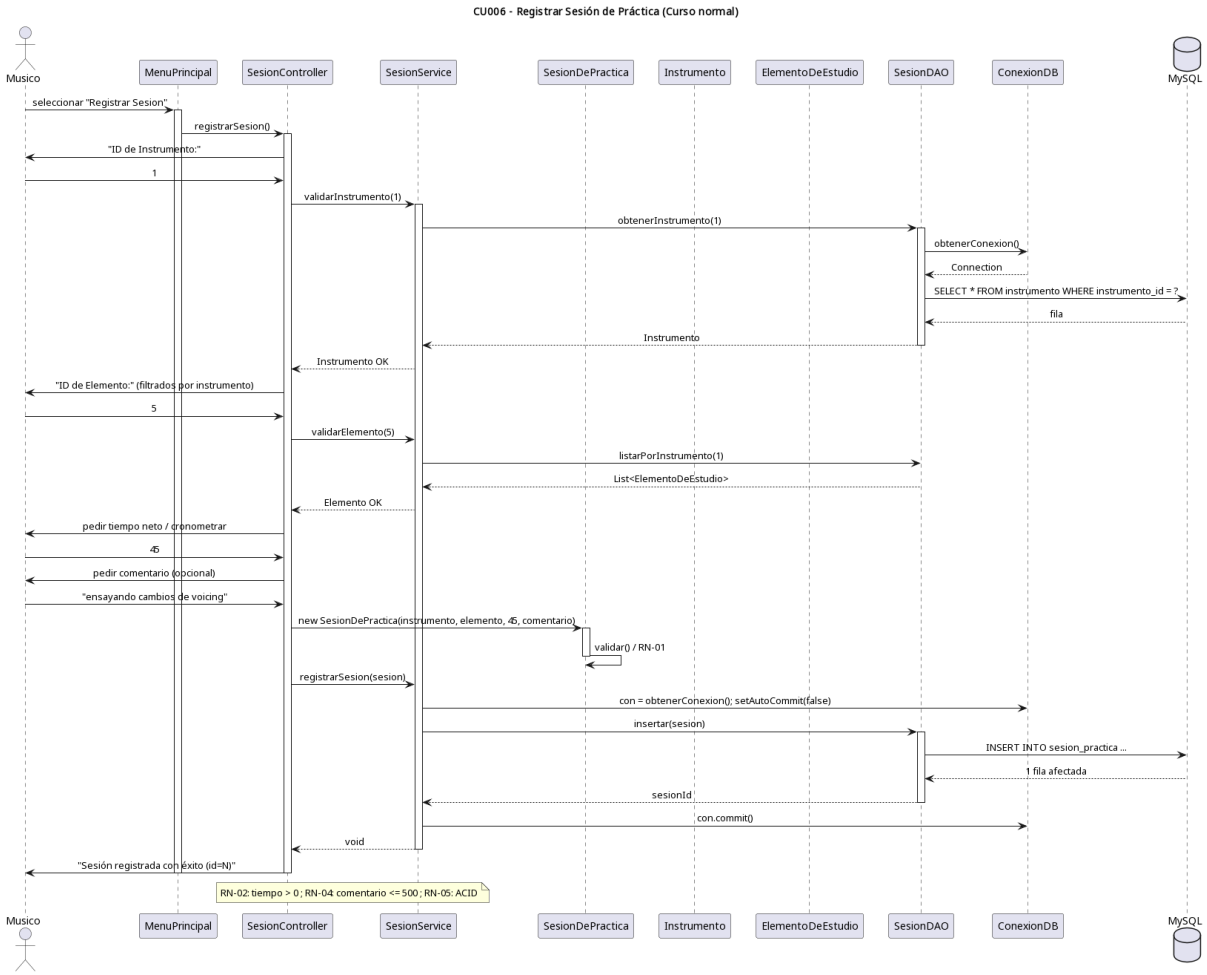


Figura 2. Diagrama de secuencia del CU006 con las clases de diseño.

8. Diagrama de clases de diseño

El diagrama se sincronizó con el código de la Entrega 4: paquete `model` (no «modelo»), asociaciones explícitas, visibilidades, tipos Java, `Estadistica` con el arreglo `int[7]`, `ExportadorTextoService`, `ArchivoException` y los enums del dominio. La figura se presenta en una página horizontal para que sea legible; el detalle de cada clase se explica en el texto.

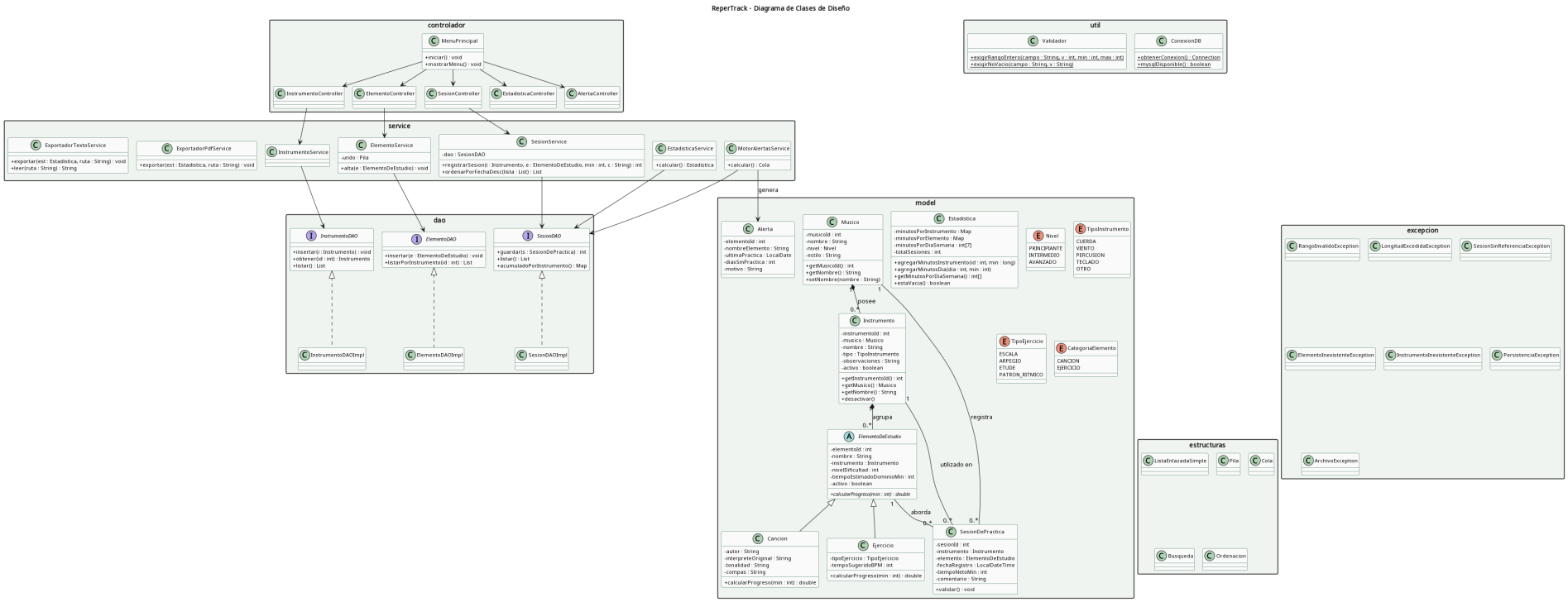


Figura 3. Diagrama de clases de diseño sincronizado con el código (Entrega 4).

Paquete	Responsabilidad
model	Entidades del dominio, enums, Estadística y Alerta.
excepcion	Excepciones checked: rango, longitud, referencia, inexistencia, persistencia y archivo.
util	Conexión JDBC y validaciones centralizadas.
dao	Interfaces e implementaciones JDBC con fallback a memoria.
service	Reglas de negocio, estadísticas, alertas y exportadores PDF/TXT.
controlador	Menú de consola y subcontroladores por módulo.
estructuras	Lista enlazada, pila, cola, búsqueda y ordenación.

9. Despliegue y aclaración del CU011

El despliegue es monousuario: la PC del músico ejecuta la aplicación sobre JRE 11 y conversa con un MySQL 8.0 local mediante mysql-connector-j sobre TCP/IP loopback :3306. No hay servidor web, ni Apache/Tomcat/HTTP/XAMPP. Los artefactos son el JAR, el script de creación, el script de integridad, el reporte TXT y el README.

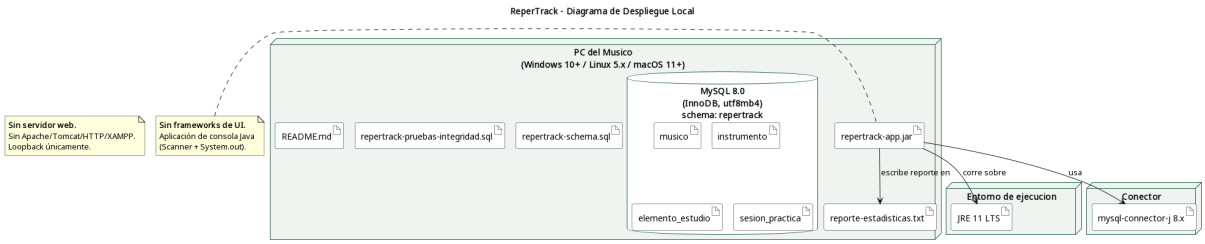


Figura 4. Diagrama de despliegue local.

CU011 — Exportar reporte. Precondiciones: hay al menos una sesión registrada y el músico eligió la opción de exportar. Postcondiciones: se genera un archivo válido en la ruta indicada y se informa el tamaño. Curso normal: se calcula la estadística, se escribe el archivo (PDF o TXT) y se muestra la confirmación. Alternativas: si no hay datos, se rechaza con un mensaje; si el archivo existe, se sobrescribe. Excepciones: error de E/S (`IOException` o `ArchivoException`) y ruta inválida. Reglas: el reporte debe contener al menos un indicador (RN-07).

10. Plan de pruebas (CP01..CP18)

La nomenclatura se unificó: el plan y los resultados usan los mismos códigos CP01..CP18. Los catorce primeros cubren sesiones, elementos, polimorfismo, estadísticas, alertas y PDF; CP15 cubre el arreglo semanal; CP16..CP18 cubren el archivo de texto (escritura/lectura, archivo inexistente y reporte vacío). Todos se ejecutan con un stub de DAO para que sean deterministas.

Código	Qué verifica	Resultado
CP01	Sesión válida persiste	PASS
CP02	Sesión huérfana rechazada (RN-01)	PASS
CP03	Tiempo cero rechazado (RN-02)	PASS
CP04	Frontera 1 min aceptada	PASS
CP05	Comentario de 501 caracteres rechazado (RN-04)	PASS
CP06	Dificultad 0/11 rechazada	PASS
CP07	BPM 19/301 rechazado	PASS
CP08	Progreso de Canción 480/960 = 0,5	PASS
CP09	Progreso de Ejercicio 120/240 = 0,5	PASS
CP10	Acumulado por instrumento (75 y 60)	PASS
CP11	Promedio global 45,0	PASS
CP12	Alerta por 45 días sin práctica	PASS
CP13	PDF válido con estadísticas	PASS
CP14	Reporte PDF vacío rechazado	PASS
CP15	Arreglo semanal de 7 días (lun=30, mié=45)	PASS
CP16	TXT se escribe y se vuelve a leer	PASS
CP17	Lectura de archivo inexistente rechazada	PASS
CP18	Reporte TXT vacío rechazado	PASS

Resultado de la corrida: 18 PASS, 0 FAIL. La evidencia completa se muestra en el Anexo B.

11. Base de datos, DER y scripts SQL

El esquema tiene cuatro tablas InnoDB en utf8mb4: musico, instrumento, elemento_estudio y sesion_practica. La herencia Canción/Ejercicio se resolvió con single-table inheritance (columna categoria + columnas específicas nulables). Las claves foráneas usan ON DELETE RESTRICT para no perder sesiones; hay UNIQUE para evitar duplicados lógicos, CHECK para las reglas de negocio e índices compuestos para las consultas de estadísticas.

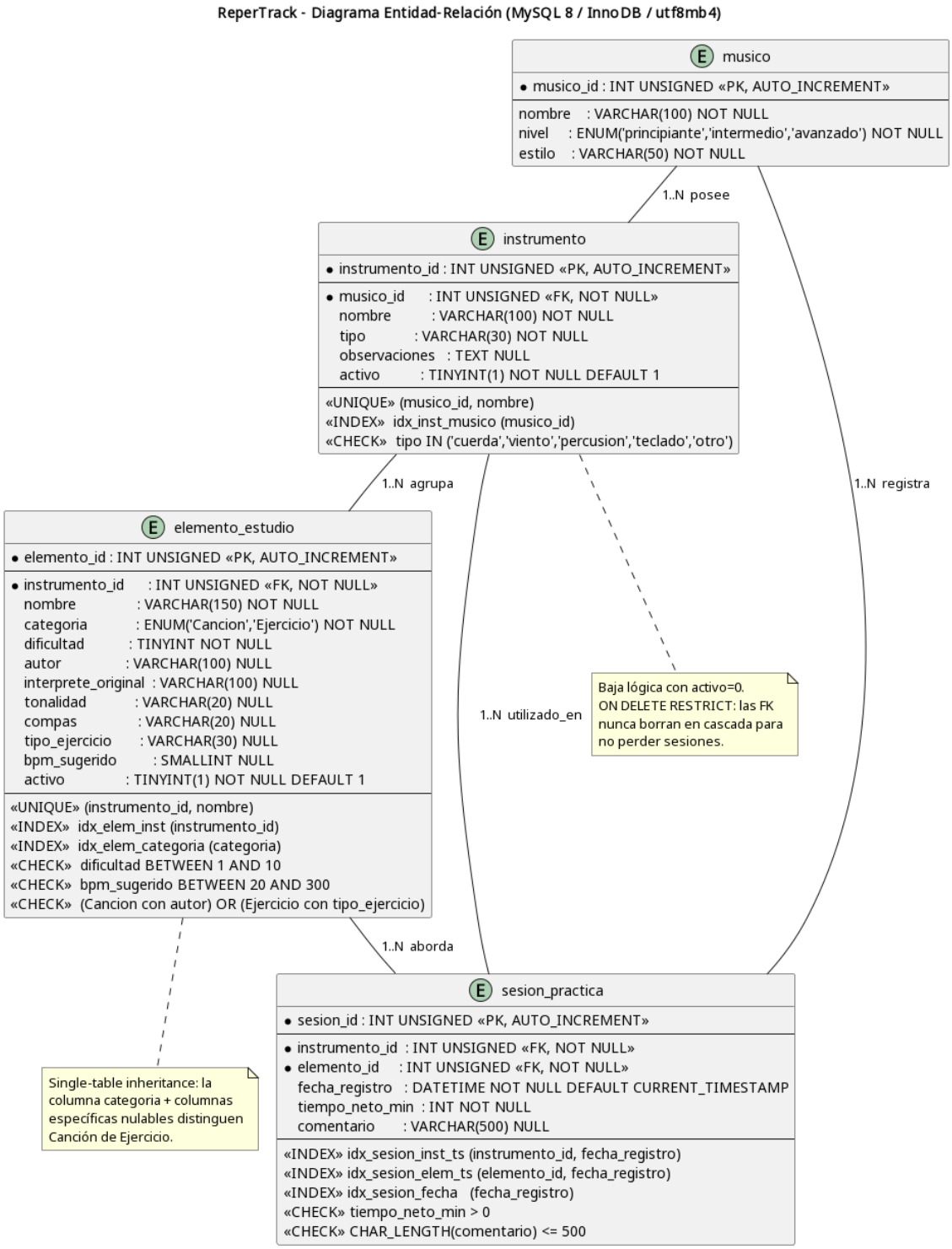


Figura 5. DER con tipos SQL explícitos (corrección del TP2).

Las diez pruebas de integridad se ejecutaron sobre MySQL 8.0.40 real (no SQLite): se rechazaron el tiempo cero, la dificultad 11, el BPM 19, la FK inexistente, el UNIQUE duplicado y el borrado con FK

activa; el INSERT válido pasó y el conteo final dio 1,1,1,1. La captura está en el Anexo B (Figura 15).

Las consultas Q1..Q5 cubren historial, acumulado por instrumento, acumulado por elemento, frecuencia/promedio y alertas. Se incluyen en el Anexo C.

12. Prototipos de consola y comunicaciones

Los prototipos de interfaz se diseñaron como pantallas ASCII: menú principal, submenú de instrumentos, submenú de elementos, registro de sesión con cronómetro, estadísticas y alertas. La implementación final conserva esa estructura y agrega las opciones 8 y 9 para el reporte TXT. Las capturas reales están en el Anexo B.

Las comunicaciones son locales: JDBC sobre TCP/IP loopback hacia MySQL; llamadas a métodos entre capas (Vista → Controlador → Servicio → DAO); sin HTTP, sin serialización y sin servidor de aplicaciones. El backup queda a cargo del músico (mysqldump).

Parte III. Trabajo Práctico 3 — Prototipo Java

El TP3 migró la vista Swing a consola pura y convirtió el modelo en un diseño con asociaciones reales. Esta parte resume los cuatro pilares y la evidencia de ejecución que el docente pidió reforzar.

13. Los cuatro pilares de la POO

Encapsulamiento. Todos los atributos del modelo son privados; el acceso se hace por getters/setters y los setters validan con `Validador`. No se puede crear una Canción con dificultad 11 sin atrapar `RangoInvalidoException`.

Herencia. `ElementoDeEstudio` es abstracta y `Cancion` y `Ejercicio` la especializan. Cada subclase redefine `calcularProgreso(int)`.

Polimorfismo. El servicio invoca `calcularProgreso` sobre la referencia del tipo padre y la JVM despacha la versión concreta: Canción divide por dificultad × 120; Ejercicio, por dificultad × 60. CP08 y CP09 lo verifican.

Abstracción. La clase abstracta define el contrato y las interfaces DAO definen la persistencia sin atar a una tecnología. El TestRunner inyecta un stub de `SesionDAO` sin tocar MySQL.

14. Excepciones y evidencia de ejecución

Hay seis excepciones checked: `RangoInvalidoException`, `LongitudExcedidaException`, `SesionSinReferenciaException`, `ElementoInexistenteException`, `InstrumentoInexistenteException` y `PersistenciaException`. La Entrega 4 agrega `ArchivoException` para el manejo de archivos. Cada controlador captura y muestra un mensaje claro; los DAO envuelven `SQLException` en `PersistenciaException`.

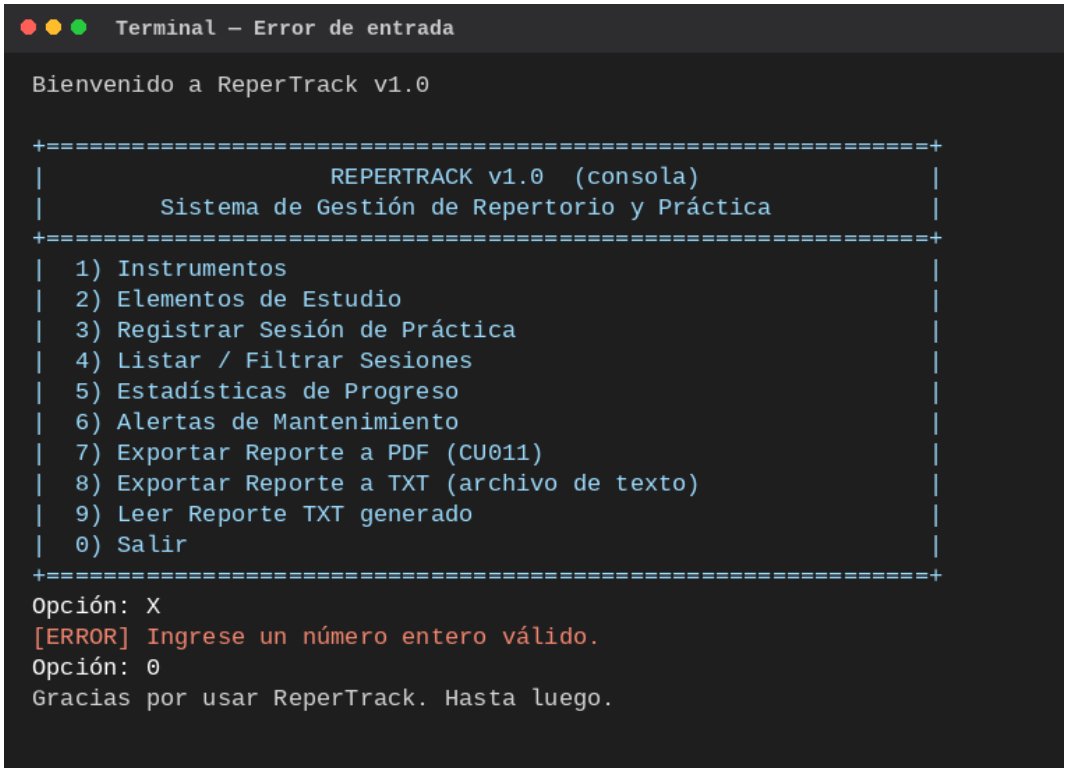


Figura B.7. Manejo de entrada inválida: se ingresa una letra donde va un número.

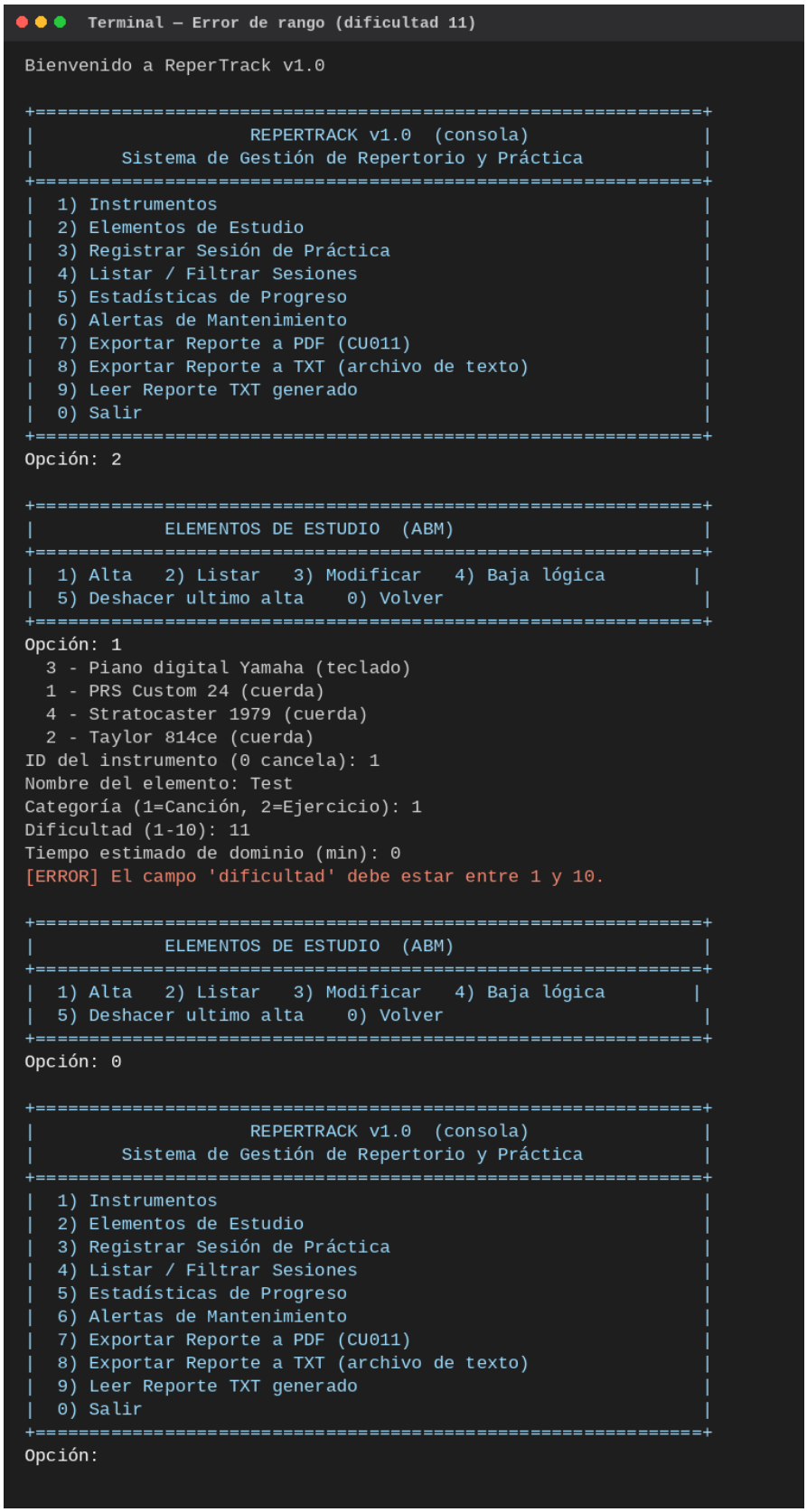


Figura B.8. Excepción de rango: dificultad 11 rechazada por la regla de negocio.

15. Menú, estructuras propias y persistencia JDBC

El menú principal tiene nueve opciones y despacha con un switch. Las estructuras propias son lista enlazada simple, pila y cola, más búsqueda lineal y ordenación por selección/burbuja. La persistencia usa JDBC con SQL parametrizado y transacciones ACID; si MySQL no responde, los DAO caen a memoria y lo informan por consola.

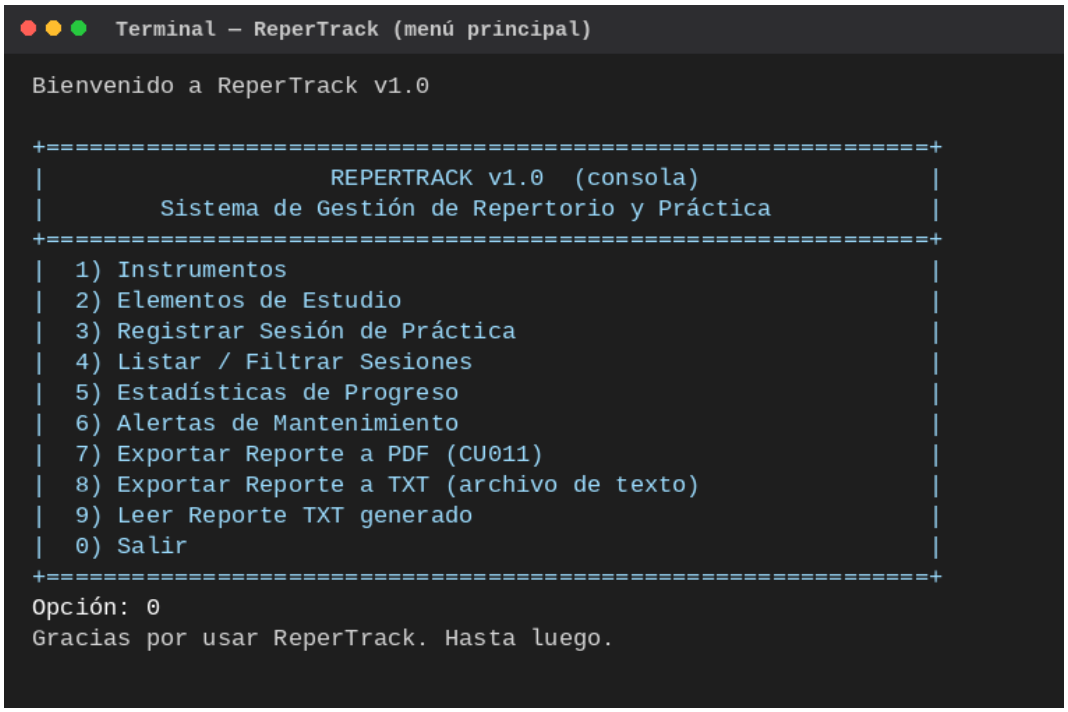


Figura B.1. Menú principal del sistema en consola.

Parte IV. Trabajo Práctico 4 — Entrega integradora

Esta parte responde a los requisitos específicos de la Entrega 4: patrón de diseño justificado, arreglos y ArrayList, archivos, excepciones en MySQL, clases abstractas/interfaces y evidencia de ejecución. También consolida las correcciones de las entregas anteriores.

16. Patrón de diseño MVC

Se aplica Modelo-Vista-Controlador. El Modelo son las entidades de `model` y los DAO; no conocen la consola ni JDBC. La Vista es la capa de controladores de consola: lee con Scanner, valida la forma de la entrada y formatea la salida; no decide reglas de negocio. El Controlador son los servicios: orquestan el flujo, aplican RN-01..RN-05 y manejan transacciones.

La separación se justifica por tres razones concretas: (1) permite probar la lógica sin levantar la interfaz (el TestRunner usa un stub de DAO); (2) aísla el cambio de presentación (pasamos de Swing en TP2 a consola en TP3 sin tocar servicios ni DAO); (3) evita que una regla de negocio quede duplicada en la vista. Si mañana se agregara una API REST, los servicios y el modelo se reutilizan tal cual.

17. Arreglos y ArrayList

La consigna pide usarlos de forma complementaria, no elegir uno. El `ArrayList` se usa para lo dinámico: los DAO devuelven `List<SesionDePractica>`, `List<Instrumento>` y `List<ElementoDeEstudio>`, y el TestRunner y los controladores los recorren con for-each. El arreglo fijo se usa para lo que tiene tamaño conocido: la distribución de minutos por día de la semana es un `int[7]` (0=lunes … 6=domingo) que se llena desde las sesiones y se muestra como barras.

```
private final int[] minutosPorDiaSemana = new int[7];

public void agregarMinutosDia(int diaSemana, int minutos) {
    if (diaSemana >= 1 && diaSemana <= 7)
        minutosPorDiaSemana[diaSemana - 1] += minutos;
}

public int[] getMinutosPorDiaSemana() { return minutosPorDiaSemana.clone(); }
```

La complementariedad es real: el ArrayList trae las filas de MySQL; el arreglo las agrega en siete posiciones fijas. CP15 verifica que el arreglo tenga siete posiciones y que la suma por día sea correcta (lunes 30, miércoles 45).

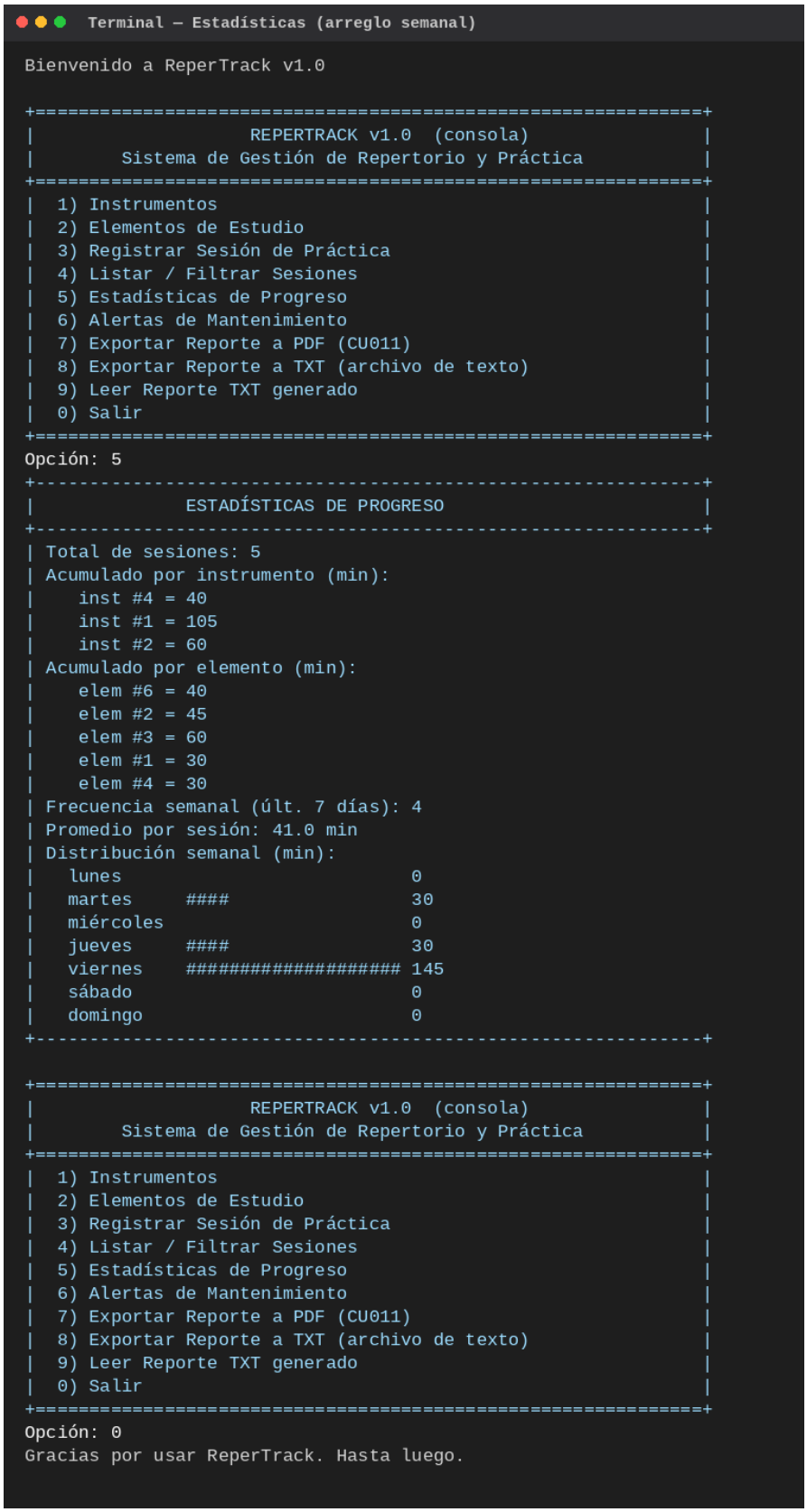


Figura B.5. Estadísticas con la distribución semanal calculada sobre el arreglo int[7].

18. Manejo de archivos de texto

Además del PDF, el sistema exporta y vuelve a leer un reporte de texto UTF-8. `ExportadorTextoService.exportar` escribe con `BufferedWriter` y `Files.newBufferedWriter`; `leer` usa `BufferedReader`. Ambos cierran el recurso con try-with-resources y envuelven `IOException` en `ArchivoException`.

```
try (BufferedWriter w = Files.newBufferedWriter(destino, StandardCharsets.UTF_8)) {
    w.write("ReperTrack - Reporte de Estadísticas"); w.newLine();
    w.write("Total de sesiones: " + est.getTotalSesiones()); w.newLine();
    // ... acumulados y distribución semanal ...
} catch (IOException ex) {
    throw new ArchivoException("No se pudo escribir el reporte de texto", ex);
}
```

La funcionalidad se expone en el menú (opciones 8 y 9) y se verifica con CP16 (escribe y lee), CP17 (archivo inexistente) y CP18 (reporte vacío). La Figura B.6 muestra la salida real.

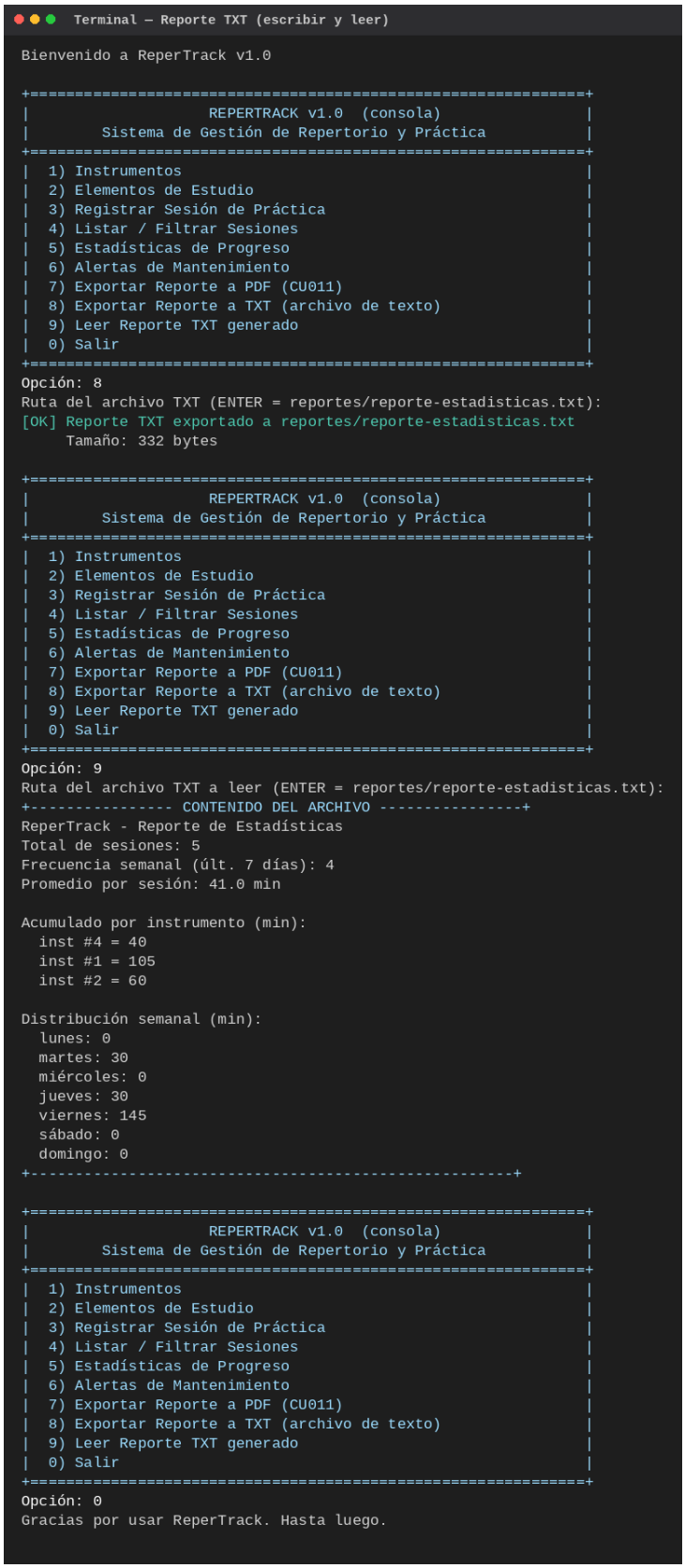


Figura B.6. Exportación y lectura del reporte TXT desde el menú.

19. Excepciones en la interacción con MySQL

Cada operación JDBC está dentro de try-with-resources. Las escrituras usan `setAutoCommit(false)`, `commit()` y `rollback()`; cualquier `SQLException` se envuelve en `PersistenciaException` para que la vista no dependa de `java.sql`. En la lectura, un `ResultSet` inválido se reporta con el mismo tipo.

La evidencia real se generó con MySQL 8.0.40 en `127.0.0.1:3307`: se creó el esquema con `01_creacion.sql`, se cargaron datos con `02_inserts.sql`, se ejecutó el programa con REPER_DB_URL apuntando al servidor y se verificó la fila con un SELECT. La Figura B.9 muestra la sesión 5 con 40 minutos recién insertada; la Figura B.10, las pruebas de integridad con los rechazos esperados.



Figura B.9. Verificación en MySQL de la sesión insertada por el programa.

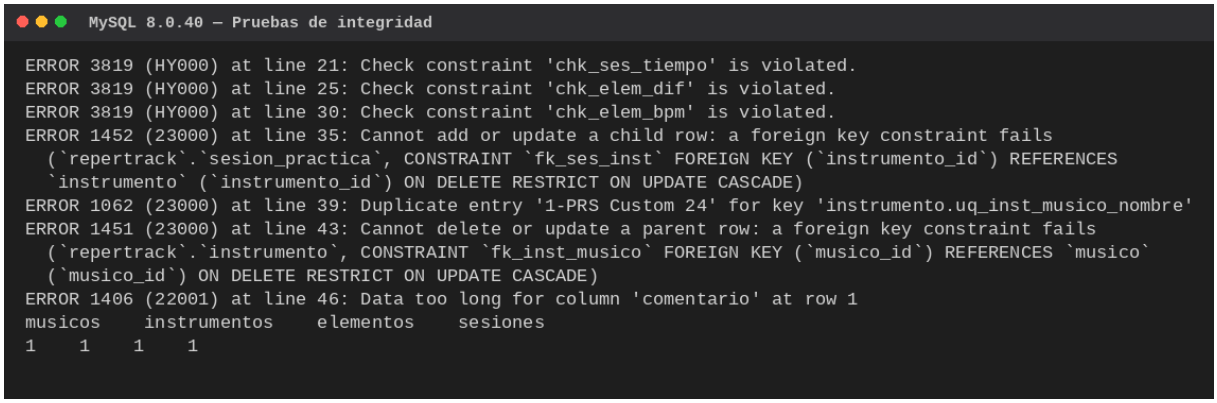


Figura B.10. Pruebas de integridad en MySQL: CHECK, FK y UNIQUE rechazan los casos inválidos.

20. Clases abstractas e interfaces

Se usan ambas y se justifica por qué. Clase abstracta: `ElementoDeEstudio` concentra los atributos comunes y obliga a implementar `calcularProgreso`; no tiene sentido instanciarla porque un «elemento genérico» no existe en el dominio. Interfaces: `InstrumentoDAO`, `ElementoDAO` y `SesionDAO` definen el contrato de persistencia y permiten inyectar un stub en las pruebas; esa es una costura que se usa de verdad, no una interfaz decorativa.

No se creó una interfaz para `ExportadorTextoService` porque tiene una sola implementación y ningún test necesita sustituirla. Si en una iteración futura se agrega CSV o JSON con el mismo contrato, ese es el disparador para extraer la interfaz.

21. Lógica de negocio y reglas RN-01..RN-05

La lógica se concentra en los servicios, no en la vista. RN-01: toda sesión debe tener instrumento y elemento; se valida en `SesionDePractica.validar()` y en `SesionService`. RN-02: el tiempo neto debe ser mayor a cero. RN-03: la fecha no puede ser futura (validada al registrar). RN-04: el comentario no supera 500 caracteres. RN-05: atomicidad transaccional en el DAO.

El flujo completo es: la vista pide los datos, el controlador los convierte a objetos, el servicio aplica las reglas y el DAO persiste. Si una regla falla, la excepción checked obliga al controlador a mostrar el mensaje y no se ejecuta el INSERT. Esa separación es la que permite que CP01..CP07 prueben las reglas sin abrir el menú.

22. Capturas del programa y video

Las capturas del Anexo B son la salida real del programa ejecutado contra MySQL: menú, alta de instrumento, alta de elemento, registro de sesión, estadísticas con el arreglo semanal, reporte TXT, errores de entrada/rango y verificación en la base. No son mockups: se capturaron desde un pseudo-terminal y se renderizaron como imagen para el PDF.

El video de presentación (3 minutos) muestra el mismo recorrido: menú, alta de instrumento, alta de obra, registro de sesión, SELECT en MySQL y estadísticas. El guion completo está en el Anexo D. Enlace del video: [PEGAR AQUÍ EL ENLACE DEL VIDEO].

23. Correcciones aplicadas y cierre

TP1: objetivos reescritos, dominio con asociaciones y atributos, propuesta técnica en consola, requerimientos con redacción revisada. TP2: secuencia de diseño con las clases del diagrama, asociación en ElementoDeEstudio, despliegue completo, CU011 especificado, CP01..CP18 unificados, DER con tipos SQL e integridad en MySQL. TP3: lógica de negocio explicada, capturas de ejecución, excepciones con evidencia y menú documentado.

El resultado es un prototipo que compila, ejecuta y pasa 18/18 pruebas; un documento que explica las decisiones y las muestra en pantalla; y un repositorio público con el código, los scripts y los diagramas. Quedan como trabajo futuro las notificaciones push, la sincronización en la nube y la planificación semanal con metas configurables, todas ya listadas como requerimientos candidatos.

24. Participación activa

Durante el cursado participé en los foros de la materia y en los encuentros sincrónicos, aporté en los espacios comunes y mantuve comunicación con el equipo docente. Las evidencias (capturas de las participaciones y enlaces a los hilos) se incorporan a continuación.

[ADJUNTAR AQUÍ LAS CAPTURAS O ENLACES DE PARTICIPACIÓN]

Anexo A. Diagrama de clases completo

Los cinco diagramas del PUD están incluidos en las Partes I y II con su explicación. Este anexo agrega la vista completa del diagrama de clases de diseño, que por su tamaño se presenta en página horizontal; el detalle de cada paquete está en la tabla siguiente y en el texto de la sección 8.

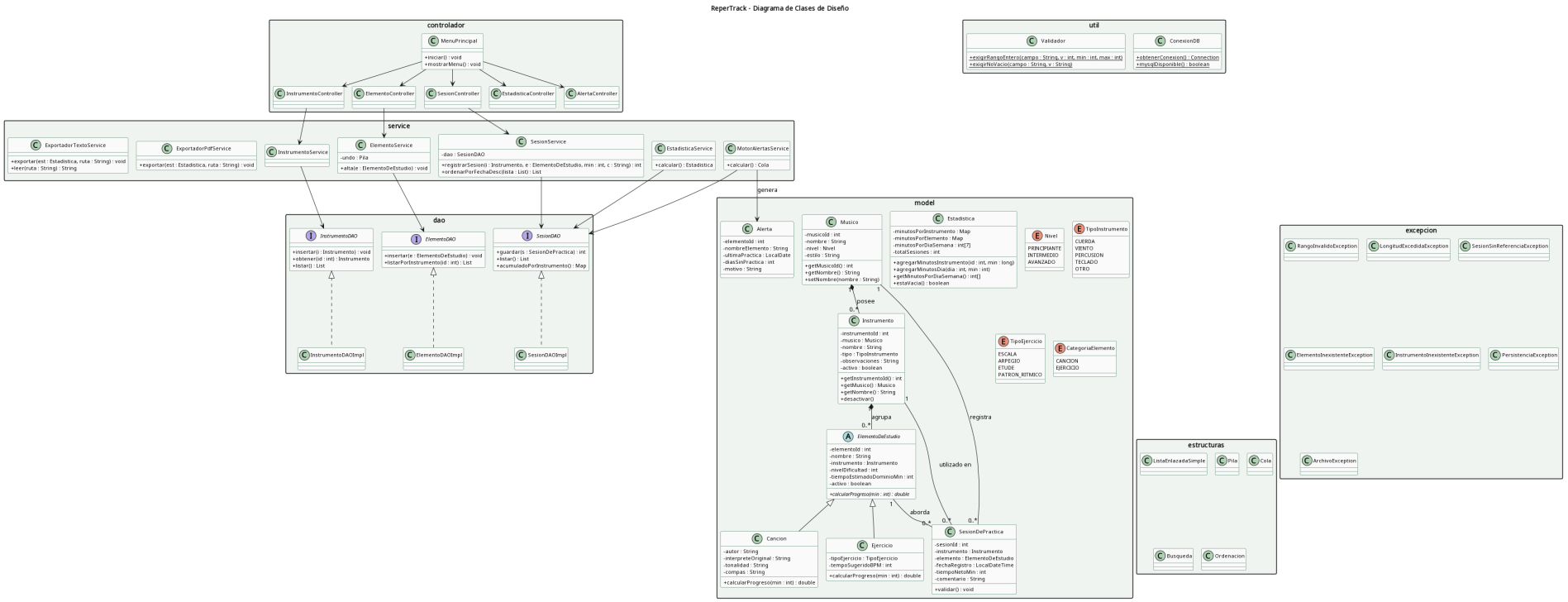


Figura A.1. Diagrama de clases de diseño sincronizado con el código.

Paquete	Responsabilidad
model	Entidades del dominio, enums, Estadística y Alerta.
excepcion	Excepciones checked: rango, longitud, referencia, inexistencia, persistencia y archivo.
util	Conexión JDBC y validaciones centralizadas.
dao	Interfaces e implementaciones JDBC con fallback a memoria.
service	Reglas de negocio, estadísticas, alertas y exportadores PDF/TXT.
controlador	Menú de consola y subcontroladores por módulo.
estructuras	Lista enlazada, pila, cola, búsqueda y ordenación.

Anexo B. Capturas de ejecución

Todas las capturas son la salida real del programa ejecutado contra MySQL 8.0.40. Se renderizaron con un marco de terminal para facilitar la lectura en el PDF.

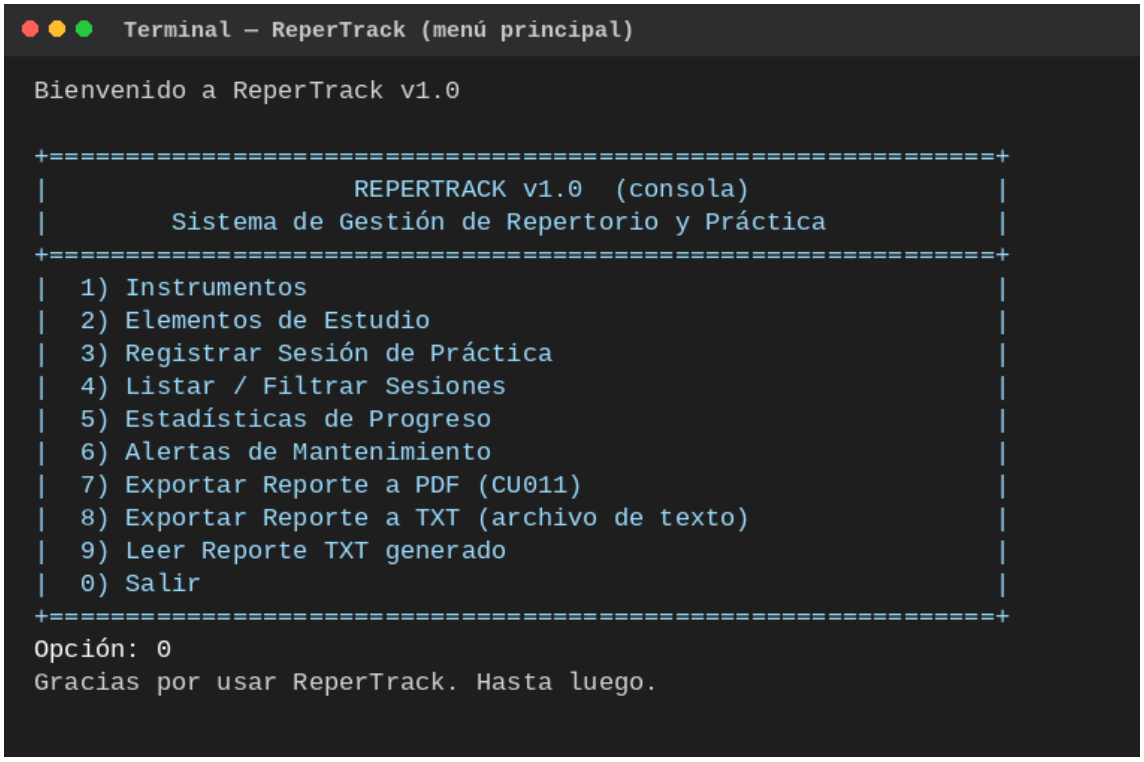


Figura B.1. Menú principal.

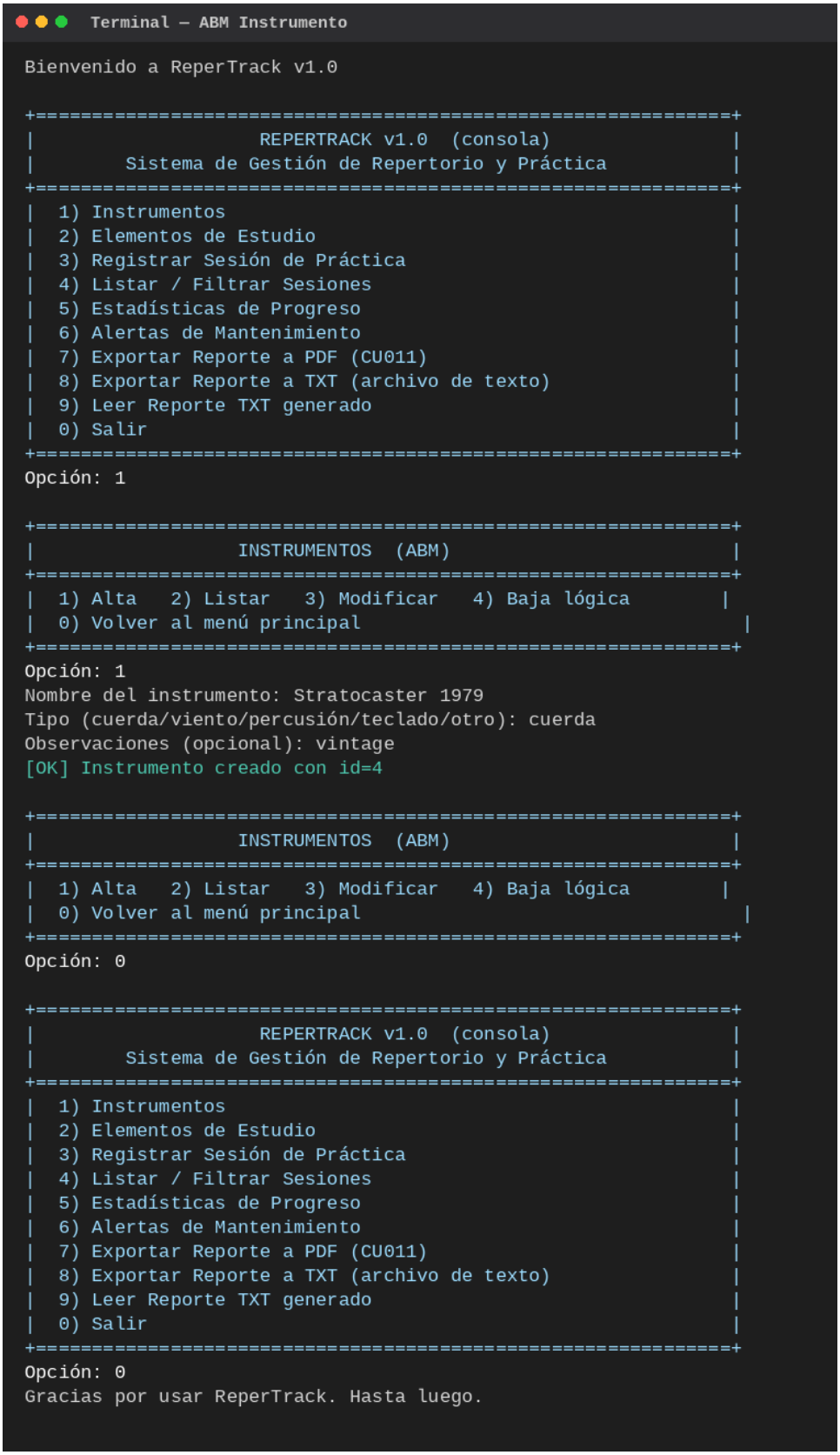


Figura B.2. Alta de instrumento.

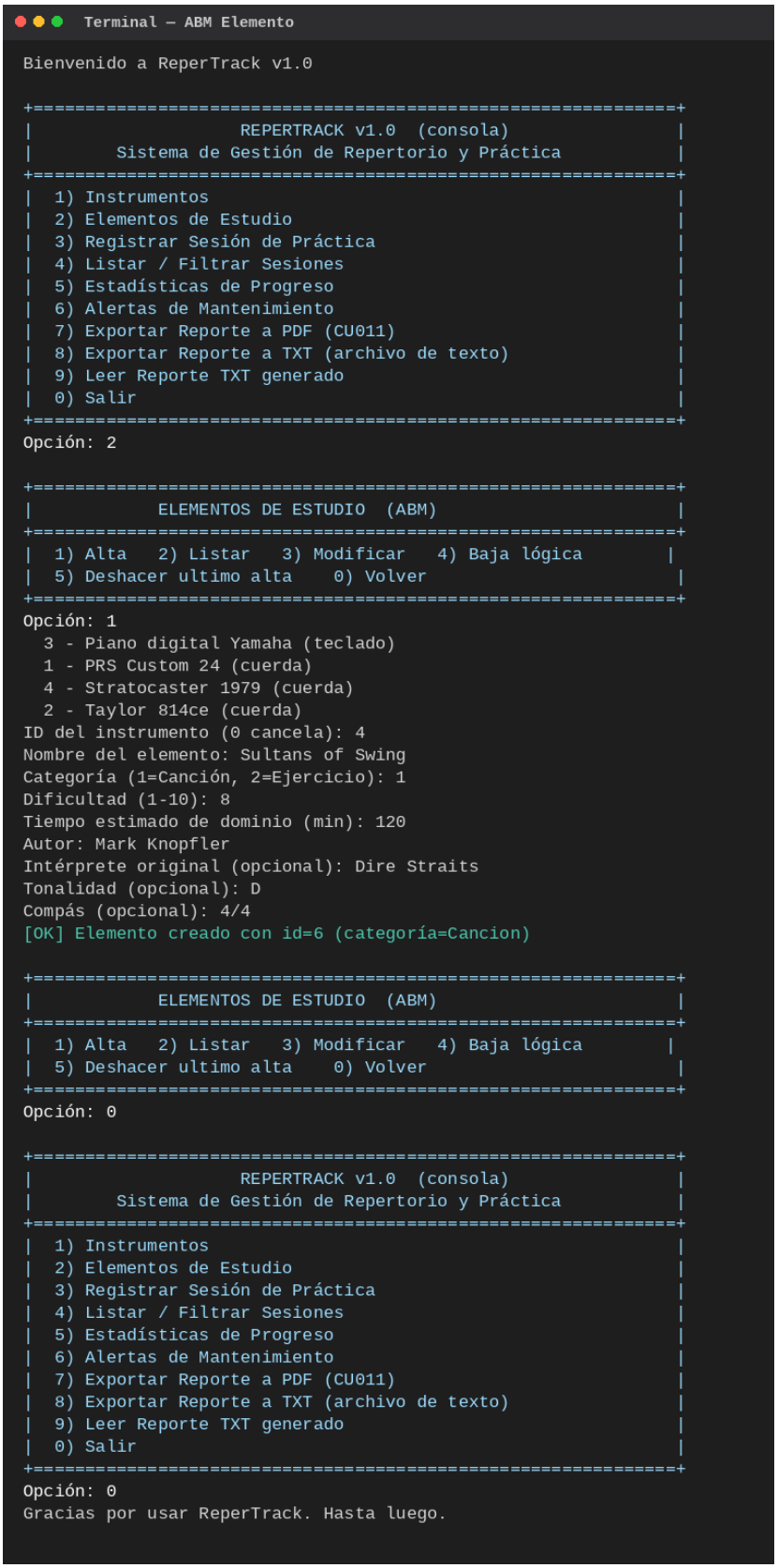


Figura B.3. Alta de elemento (canción).

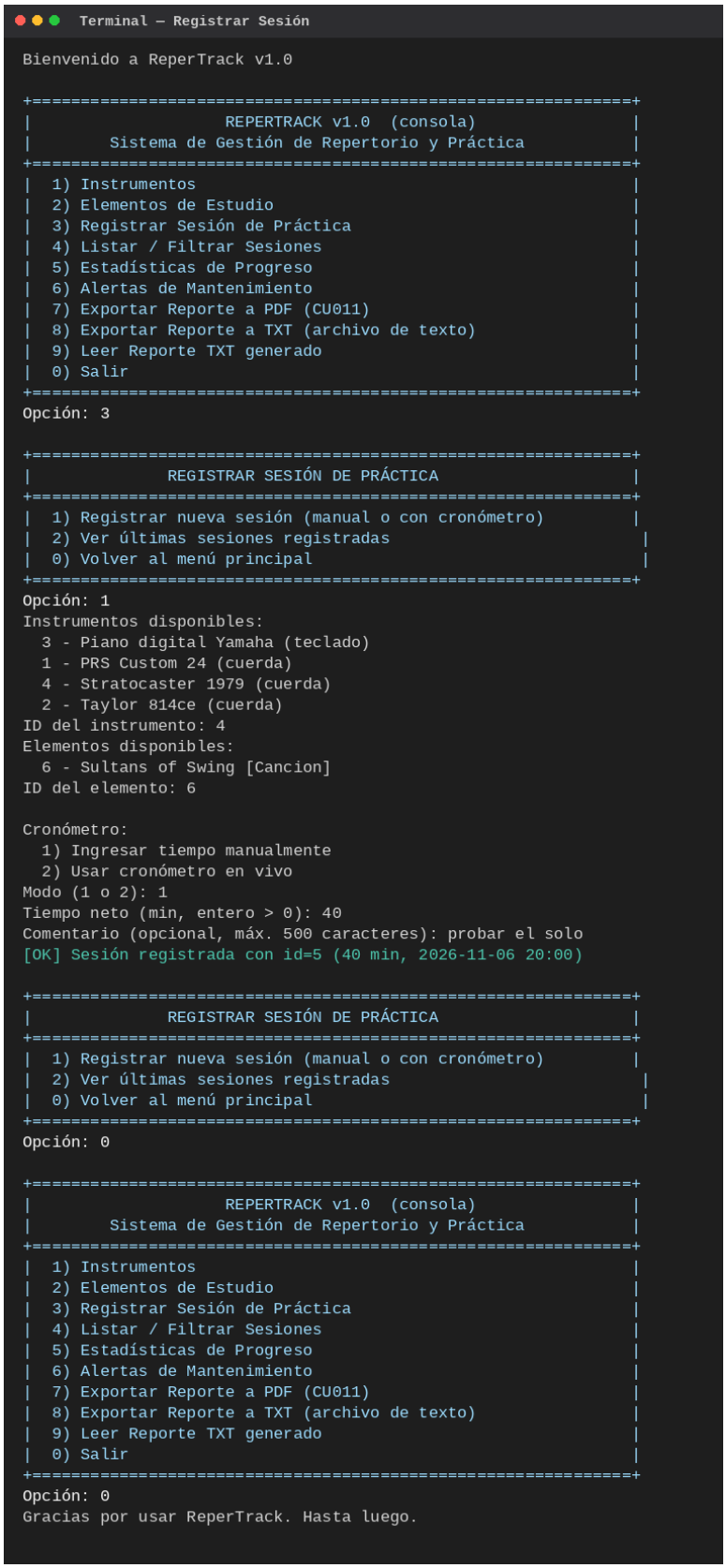


Figura B.4. Registro de sesión con instrumento y elemento.

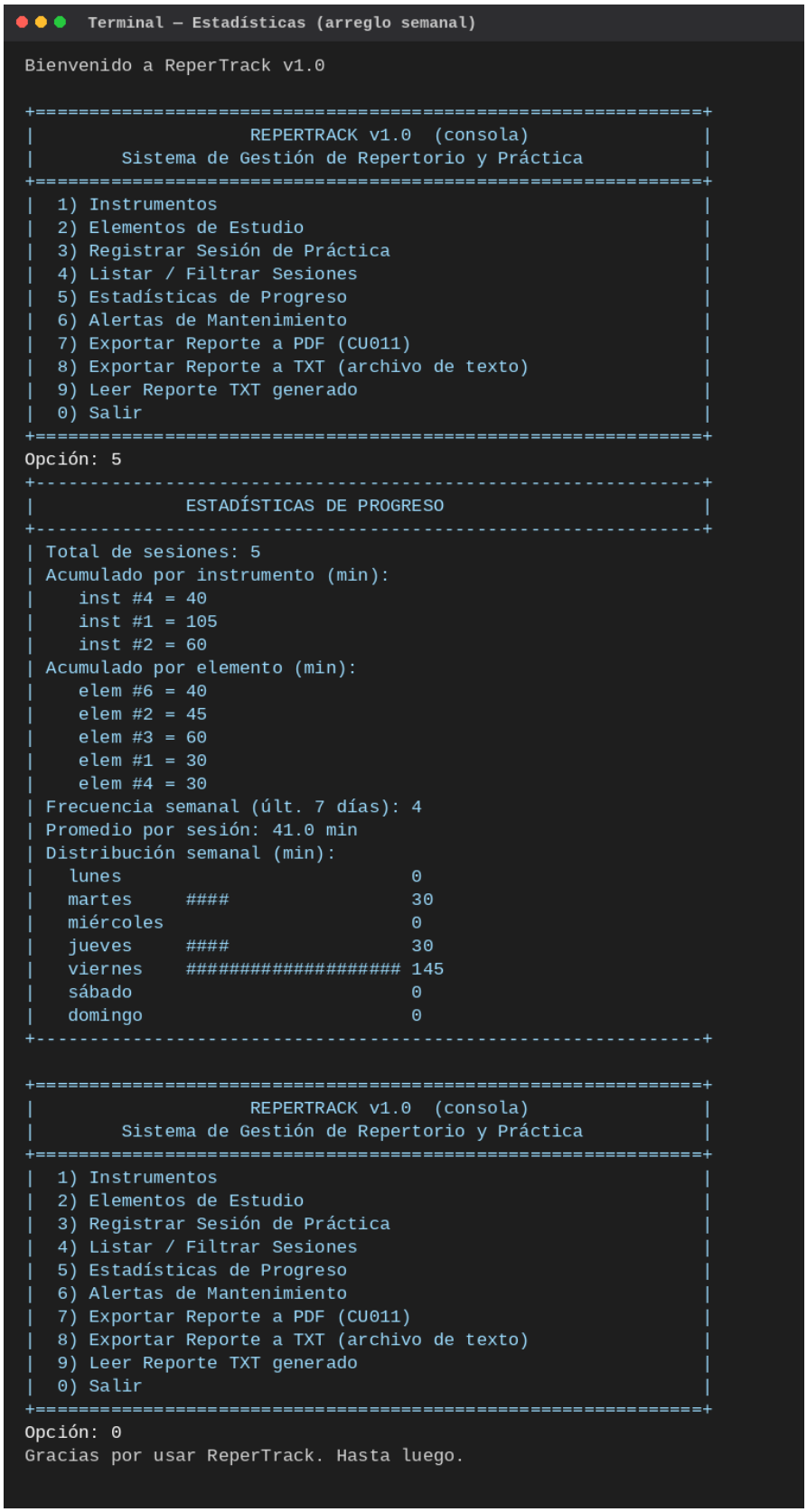


Figura B.5. Estadísticas y distribución semanal.

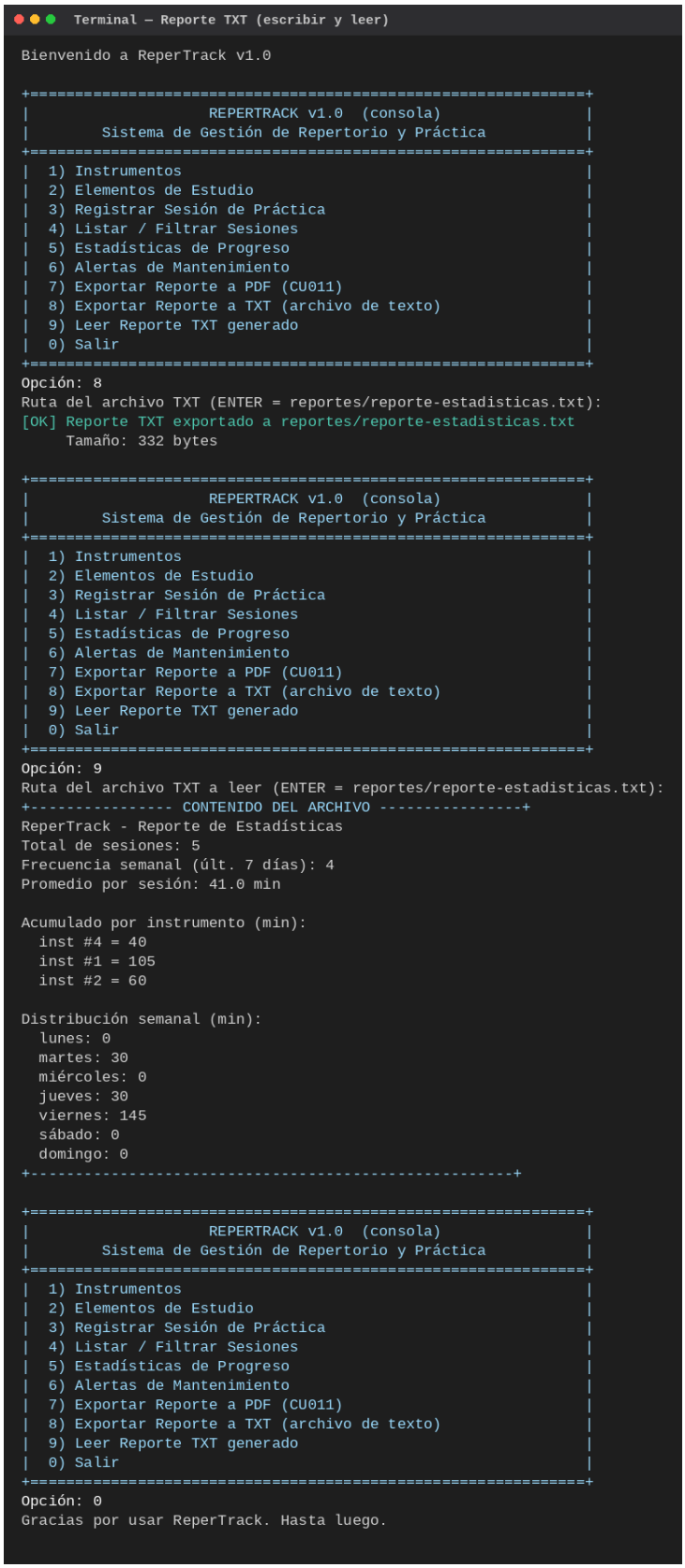


Figura B.6. Reporte TXT: escritura y lectura.

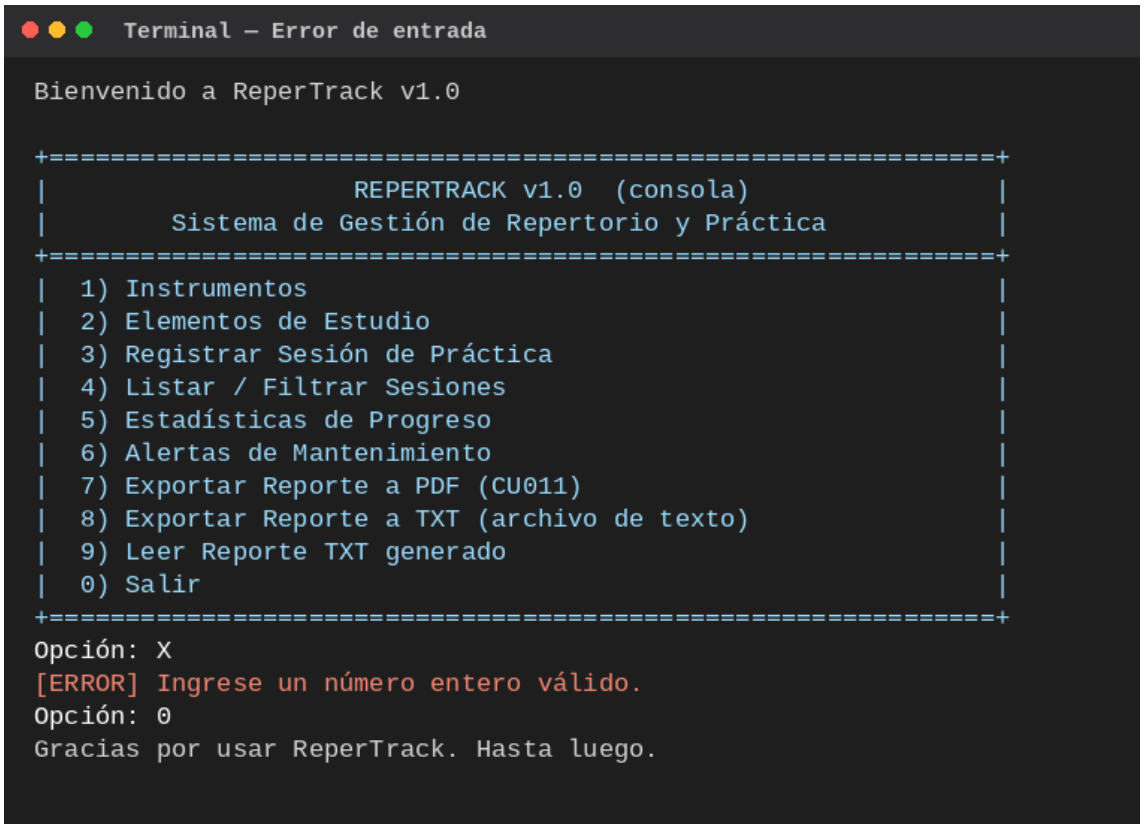


Figura B.7. Error de entrada inválida.

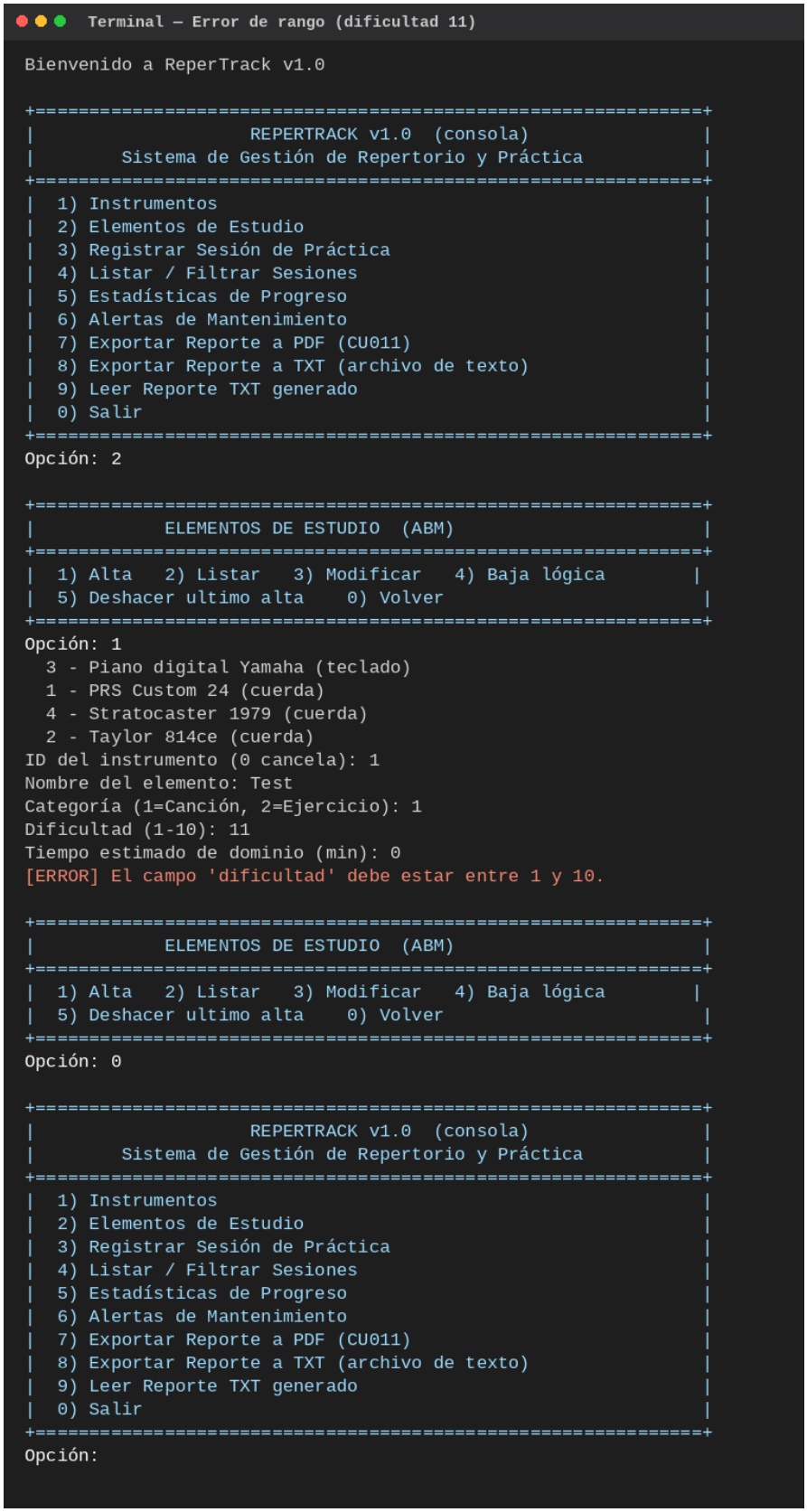


Figura B.8. Error de rango en la dificultad.

```
MySQL 8.0.40 — SELECT de verificación

+----------------+----------------------+---------+--------+
| instrumento_id | nombre               | tipo    | activo |
+----------------+----------------------+---------+--------+
|              1 | PRS Custom 24        | cuerda  |      1 |
|              2 | Taylor 814ce         | cuerda  |      1 |
|              3 | Piano digital Yamaha | teclado |      1 |
|              4 | Stratocaster 1979    | cuerda  |      1 |
+----------------+----------------------+---------+--------+
+-----------+----------------+-------------+-----------------+---------------------+
| sesion_id | instrumento_id | elemento_id | tiempo_neto_min | fecha_registro      |
+-----------+----------------+-------------+-----------------+---------------------+
|         5 |              4 |           6 |              40 | 2026-11-06 22:57:22 |
|         4 |              1 |           4 |              30 | 2026-09-22 19:57:06 |
|         3 |              2 |           3 |              60 | 2026-11-06 19:57:06 |
+-----------+----------------+-------------+-----------------+---------------------+
```

Figura B.9. Verificación de la fila en MySQL.

```
MySQL 8.0.40 — Pruebas de integridad

ERROR 3819 (HY000) at line 21: Check constraint 'chk_ses_tiempo' is violated.
ERROR 3819 (HY000) at line 25: Check constraint 'chk_elem_dif' is violated.
ERROR 3819 (HY000) at line 30: Check constraint 'chk_elem_bpm' is violated.
ERROR 1452 (23000) at line 35: Cannot add or update a child row: a foreign key constraint fails
  (`repertrack`.`sesion_practica`, CONSTRAINT `fk_ses_inst` FOREIGN KEY (`instrumento_id`) REFERENCES
  `instrumento` (`instrumento_id`) ON DELETE RESTRICT ON UPDATE CASCADE)
ERROR 1062 (23000) at line 39: Duplicate entry '1-PRS Custom 24' for key 'instrumento.uq_inst_musico_nombre'
ERROR 1451 (23000) at line 43: Cannot delete or update a parent row: a foreign key constraint fails
  (`repertrack`.`instrumento`, CONSTRAINT `fk_inst_musico` FOREIGN KEY (`musico_id`) REFERENCES `musico`
  (`musico_id`) ON DELETE RESTRICT ON UPDATE CASCADE)
ERROR 1406 (22001) at line 46: Data too long for column 'comentario' at row 1
musicos    instrumentos    elementos    sesiones
1    1    1    1
```

Figura B.10. Pruebas de integridad en MySQL.

Anexo C. Consultas SQL

Las consultas Q1..Q5 se ejecutan en el cliente MySQL y se corresponden con los casos de uso. Se reproducen para que el lector pueda verificarlas.

```
-- Q1. Historial completo (CU007)
SELECT SP.sesion_id, M.nombre AS musico, I.nombre AS instrumento,
       EE.nombre AS elemento, EE.categoria, SP.tiempo_neto_min AS minutos,
       SP.fecha_registro AS fecha
  FROM sesion_practica SP
  JOIN instrumento I ON SP.instrumento_id = I.instrumento_id
  JOIN musico M ON I.musico_id = M.musico_id
  JOIN elemento_estudio EE ON SP.elemento_id = EE.elemento_id
 ORDER BY SP.fecha_registro DESC;
```

```
-- Q2. Acumulado por instrumento (CU009)
SELECT I.nombre AS instrumento, SUM(SP.tiempo_neto_min) AS minutos_totales
  FROM sesion_practica SP JOIN instrumento I ON SP.instrumento_id = I.instrumento_id
 GROUP BY I.nombre;
```

```
-- Q3. Acumulado por elemento (CU009)
SELECT EE.nombre AS elemento, SUM(SP.tiempo_neto_min) AS minutos_totales
  FROM sesion_practica SP JOIN elemento_estudio EE ON SP.elemento_id = EE.elemento_i
d
 GROUP BY EE.nombre;
```

```
-- Q4. Frecuencia y promedio (RFS09)
SELECT COUNT(*) AS sesiones_7d, AVG(tiempo_neto_min) AS promedio_min
  FROM sesion_practica WHERE fecha_registro >= NOW() - INTERVAL 7 DAY;
```

```
-- Q5. Alertas de mantenimiento (CU012)
SELECT EE.elemento_id, EE.nombre, MAX(SP.fecha_registro) AS ultima,
       DATEDIFF(NOW(), MAX(SP.fecha_registro)) AS dias
  FROM elemento_estudio EE LEFT JOIN sesion_practica SP ON SP.elemento_id = EE.eleme
nto_id
 WHERE EE.activo = 1 GROUP BY EE.elemento_id
HAVING ultima IS NULL OR DATEDIFF(NOW(), ultima) > 30;
```

Anexo D. Guion del video

El video dura 3 minutos y muestra el core del sistema contra MySQL. El guion completo está en `docs/guion-video.md`; aquí se resume el recorrido.

- 0:00–0:15 — Presentación del proyecto y del stack (Java 11 + MySQL 8 + JDBC).
- 0:15–0:40 — Menú y arquitectura MVC.
- 0:40–1:15 — Alta de un instrumento.
- 1:15–1:50 — Alta de una canción asociada.
- 1:50–2:30 — Registro de una sesión y verificación con SELECT en MySQL.
- 2:30–2:50 — Estadísticas con el arreglo semanal y exportación TXT.
- 2:50–3:00 — Cierre y enlace al repositorio.

Enlace del video: [PEGAR AQUÍ EL ENLACE DEL VIDEO ANTES DE ENTREGAR]

Referencias bibliográficas

Braude, E. (2003). Ingeniería de software: una perspectiva orientada a objetos. Alfaomega.

Ericsson, K. A., Krampe, R. T., y Tesch-Römer, C. (1993). The role of deliberate practice in the acquisition of expert performance. Psychological Review, 100(3), 363-406.

Institute of Electrical and Electronics Engineers. (2008). Standard for Software Quality Assurance Plans (ANSI/IEEE Standard No. 829).

Jacobson, I., Booch, G., y Rumbaugh, J. (1999). The unified software development process. Addison-Wesley.

Kendall, K., y Kendall, J. (2011). Análisis y diseño de sistemas. Pearson Educación.

Newport, C. (2016). Deep work: Rules for focused success in a distracted world. Grand Central Publishing.

Reinosa, E. J., Maldonado, C. A., Muñoz, R., Damiano, L. E., y Abrutsky, M. A. (2012). Bases de datos. Alfaomega.

Rumbaugh, J., Jacobson, I., y Booch, G. (2006). El lenguaje unificado de modelado (2.ª ed.). Pearson-Addison-Wesley.

Saroka, R. (2002). Sistemas de información en la era digital. Fundación OSDE.

Sommerville, I. (2005). Ingeniería del software (7.ª ed.). Pearson-Addison-Wesley.

Stair, R., y Reynolds, G. (2010). Principios de sistemas de información (9.ª ed.). Cengage Learning.

Repositorio público del proyecto: https://github.com/duendecillo2/reper-track